LEARNING ROADMAP · DESIGN VERIFICATION

From Zero to Design Verification Engineer

Electronics → digital design → SystemVerilog → UVM → real projects

This roadmap takes you from zero to a working Design Verification (DV) skill set: digital logic first, then Verilog/SystemVerilog, then UVM, then real projects. Each phase lists what to watch, what to install and how to tell you are ready to move on. **Build something in every phase**; watching alone does not build the skill.



Read this first: KiCad and Cadence are different kinds of tool

- **KiCad** is for printed circuit board (PCB) design. It builds hardware intuition, but it is not on the DV path, so treat it as optional.
- **Cadence** covers two worlds: board tools (OrCAD, Allegro) and chip tools (Xcelium simulator, JasperGold formal, Virtuoso analog layout, Genus and Innovus for synthesis and place-and-route).
- **A DV engineer** mostly lives in simulators (Xcelium, Synopsys VCS, Siemens Questa) with UVM on top.

Roadmap at a glance

#	Phase	Time	You walk away with
1	**Electronics basics**	4–6 weeks	Intuition for voltage, current, RC circuits and transistors as switches.
2	**Digital design and computer architecture**	6–8 weeks	The real foundation: logic, FSMs, timing, clock domains, pipelines and caches.
3	**Verilog / SystemVerilog and a little FPGA**	6–8 weeks	RTL you can write, simulate and debug from waveforms.
4	**Actual verification**	3–4 months	SystemVerilog for verification, SVA, coverage, UVM, intro formal and CDC. This is the job.
5	**Prove it**	Ongoing	A verified FIFO, serial protocols, AXI-Lite and a RISC-V core: your portfolio.

Phases 1–4 add up to roughly 7–9 months at a steady pace. If you already have an electronics or CS background, skim Phase 1 and move quickly through Phase 2.

Two traps to avoid

- **Binge-watching without building.** Every phase should end with something you built and simulated yourself.
- **Learning UVM syntax without learning what to verify.** UVM is only machinery. The real skill is deciding what to check and how you will know you checked enough. That is why the test plan comes before the testbench; a worked example follows the phases.

The phases in detail

1 Electronics basics

Learn just enough

4–6 weeks

LEARN	Voltage and current, RC circuits, diodes, transistors as switches, and the idea of an op-amp.
WATCH	• **Khan Academy** or **MIT OCW 6.002** for theory • **All About Electronics** for intuition • **EEVblog** and **Great Scott!** for a hands-on feel
USE	• **Falstad** browser circuit simulator • **LTspice** • A breadboard and a multimeter
☐ **DONE WHEN**	You can explain how an RC circuit charges and how a transistor works as a switch, and you have simulated both.

2 Digital design and computer architecture

The real foundation

6–8 weeks

COVER	Combinational and sequential logic, FSMs, setup/hold timing, clock domains, memories, pipelines and caches.
WATCH	• **Ben Eater**: building an 8-bit computer is the best “how a CPU really works” series • **Neso Academy** • **Onur Mutlu**’s free ETH Zürich lectures • **Nand2Tetris**
USE	• **Logisim-evolution** or **Digital** (logic simulators)
☐ **DONE WHEN**	You can turn a spec into an FSM, explain setup/hold and why clock-domain crossings are risky, and describe how a simple pipelined CPU executes instructions.

3 Verilog / SystemVerilog and a little FPGA

Write RTL, simulate it, optionally run it on hardware

6–8 weeks

WATCH	• **Nandland**: Verilog, VHDL and FPGA basics • **Matt Venn’s Zero to ASIC**: RTL to real silicon with open-source tools • **NPTEL**: IIT digital design and VLSI courses
USE	• **HDLBits** for daily practice • **Icarus Verilog** with **GTKWave**, plus **Verilator** and **Yosys** • Free **Vivado** or **Quartus** editions • Optional: a cheap FPGA board (Tang Nano, iCE40 or Basys 3) to make concepts tangible
☐ **DONE WHEN**	You can write, simulate and debug a counter, an FSM and a small FIFO in Verilog, and read the waveforms to find your own bugs.

4 Actual verification

This is the job

3–4 months

SOURCES	YouTube is thinner here. The best free material is **Verification Academy** (Siemens), **ChipVerify** and **Sutherland HDL**'s papers.
LEARN IN ORDER	1. **SystemVerilog for verification**: OOP, randomization, constraints, interfaces, clocking blocks 2. **Assertions** (SVA) 3. **Functional coverage** 4. **UVM**: agents, sequences, scoreboards, the factory, config_db and phases 5. **Intro formal** and **CDC** basics
ALSO TRY	**cocotb** (Python) as a lightweight alternative for building testbenches.
PRACTICE ON	**EDA Playground** runs SystemVerilog and UVM in your browser. Sign in to use the free simulators and Aldec Riviera-PRO, which supports UVM. The Cadence, Synopsys and Siemens simulators there need a validated company or institutional account.
☐ DONE WHEN	You have built a constrained-random UVM testbench with an agent, scoreboard, functional coverage and SVA for a small design, and you can explain what each coverage hole means.

5 Prove it

Build, break and verify real designs

Ongoing

BUILD IN ORDER	1. **Sync and async FIFO**: scoreboards, reference models, full/empty checks; the async version adds clock-domain crossing 2. **UART, SPI and I2C**: drivers, monitors and agents for real protocols, plus protocol coverage 3. **AXI-Lite slave**: handshake assertions, reusable agents, constrained-random sequences 4. **Small RISC-V core** (PicoRV32 or Ibex): a coverage report, regressions, and deliberately injected bugs to prove your testbench catches them
THEN READ	**OpenTitan**'s public DV environments, to see what industrial-grade verification looks like.
☐ DONE WHEN	Each project sits in a public repository with its test plan, coverage report and a short log of the bugs your testbench found.

Write the test plan before the testbench

A test plan lists, for each feature, how you will exercise it, how you will know it worked and how you will know you tried enough. The testbench is only the machinery that carries it out. The idea, for a FIFO:

Feature	Stimulus	Check	Coverage
Data order and integrity	Random writes and reads at varying rates	Scoreboard: data comes out in the order it went in	Fill level: empty, partial, full
Write when full	Keep writing after the FIFO fills	Behaves as specified (ignored or flagged); stored data is not corrupted	Write attempted while full
Read when empty	Read from an empty FIFO	Behaves as specified; no spurious data; empty flag correct	Read attempted while empty
Reset mid-traffic	Assert reset during active reads and writes	FIFO empties and flags return to their idle values	Reset hit at each fill level

Tools: free first, industry standard later

Task	Learn with (free)	Industry standard
RTL simulation	Icarus, Verilator, EDA Playground	VCS, Xcelium, Questa
Waveform debug	GTKWave, Surfer	Verdi, SimVision/Indago, Visualizer
Formal	SymbiYosys	JasperGold, VC Formal, Questa Formal
Synthesis / P&R	Yosys, OpenROAD/OpenLane	Genus/Innovus, Design Compiler/Fusion Compiler
Analog / SPICE	LTspice, ngspice, Xschem, Magic	Virtuoso + Spectre, HSPICE
PCB	KiCad	Allegro/OrCAD, Altium

Cadence, Synopsys and Siemens licenses are expensive, so you will normally get them through a university or an employer. The open-source stack plus EDA Playground gets you most of the way. On EDA Playground the big-vendor simulators need a validated company or institutional account, so plan on Aldec Riviera-PRO there for UVM practice.

Alongside the tools

Everyday engineering skills

- Linux command line
- Git
- Python
- Tcl and Makefiles
- Basic C/C++

These matter in DV as much as UVM does.

Industry context (YouTube)

- Asianometry
- High Yield
- Branch Education
- TechTechPotato

They explain how chips are made and sold.

Books

- *Digital Design and Computer Architecture*, Harris & Harris
- *SystemVerilog for Verification*, Spear & Tumbush

Read alongside Phases 2 and 4.

Quick links

Falstad Circuit Simulator
falstad.com/circuit · Phase 1

NPTEL (IIT courses)
nptel.ac.in · Phases 2–3

HDLBits
hdlbits.01xz.net · Phase 3

ChipVerify
chipverify.com · Phase 4

KiCad
kicad.org · Optional

Nand2Tetris
nand2tetris.org · Phase 2

Nandland
nandland.com · Phase 3

EDA Playground
edaplayground.com · Phases 3–4

Sutherland HDL
sutherland-hdl.com · Phase 4

Ben Eater: 8-bit computer
eater.net/8bit · Phase 2

Zero to ASIC Course
zerotoasiccourse.com · Phase 3

Verification Academy
verificationacademy.com · Phase 4

OpenTitan
opentitan.org · Phase 5

Sites move and free-tier terms change. If a link no longer works, search for the name. YouTube channels mentioned in the phases are best found by searching their names.